Atividade Prática: Prototype

1. Contexto

Uma distribuidora de alimentos atende supermercados, padarias e restaurantes. Todo mês, cada cliente faz um pedido de reposição quase idêntico ao do mês anterior: os mesmos produtos e quantidades, mudando apenas o cliente, o endereço de entrega e alguns itens. Hoje o atendente monta cada pedido do zero, item por item, o que consome tempo e gera erros de digitação.

A distribuidora quer cadastrar pedidos-modelo e gerar os novos pedidos a partir da cópia desses modelos.

2. Objetivo

Implementar em Java o sistema de pedidos da distribuidora aplicando o padrão Prototype. A cópia de um pedido deve ser profunda: alterar um pedido gerado não pode modificar o modelo nem os demais pedidos.

3. Requisitos funcionais

ID	Requisito	Nível
RF01	O sistema deve definir a interface genérica Prototype<T>, com o método T clonar().	Básico
RF02	O sistema deve representar cada linha de pedido na classe ItemPedido, com os atributos produto (String), quantidade (int) e precoUnitario (BigDecimal), e fornecer o método subtotal(), que retorna quantidade multiplicada pelo preço unitário.	Básico
RF03	A classe ItemPedido deve possuir um construtor de cópia, ItemPedido(ItemPedido origem), que produza um item independente do original.	Básico
RF04	O sistema deve representar o pedido na classe Pedido, com os atributos id (long), cliente (String), enderecoEntrega (String), itens (List<ItemPedido>) e criadoEm (LocalDate), e fornecer o método valorTotal(), que soma os subtotais dos itens com duas casas decimais.	Intermediário
RF05	O sistema deve atribuir a cada Pedido, no momento de sua criação, um id sequencial único, iniciando em 1.	Intermediário
RF06	A classe Pedido deve implementar Prototype<Pedido>. O método clonar() deve gerar uma cópia profunda: cliente e endereço copiados, e a lista de itens da cópia deve ser uma nova lista com novos objetos ItemPedido.	Intermediário
RF07	A cópia de um Pedido deve receber um novo id (RF05) e a data de criação do momento da cópia. O id e a data do original não devem ser copiados.	Intermediário
RF08	O sistema deve oferecer a classe CatalogoDePedidos, com os métodos registrar(String chave, Pedido modelo) e criar(String chave). O método criar deve devolver sempre uma cópia do modelo, nunca o próprio objeto armazenado.	Intermediário

ID	Requisito	Nível
RF09	O método criar de CatalogoDePedidos deve lançar IllegalArgumentException, com mensagem que informe a chave, quando ela não estiver registrada.	Intermediário
RF10	O sistema deve conter a classe PedidosApp, com o método main, que execute o cenário de teste da seção 5 e imprima no console cada pedido no formato: Pedido #id | cliente | endereço | itens=N | total=R$ valor.	Avançado
RF11	A classe PedidosApp deve verificar automaticamente os critérios de aceitação da seção 6 e imprimir, para cada critério, a linha "CA-xx descrição: OK" ou "CA-xx descrição: FALHA".	Avançado
RF12	(Desafio) A classe Pedido deve oferecer também o método clonarPorSerializacao(), que respeite os mesmos requisitos RF06 e RF07. O PedidosApp deve medir e imprimir, em milissegundos, o tempo de 100.000 cópias por clonar() e por clonarPorSerializacao().	Desafio

4. Restrições técnicas

RT01. O código deve compilar com javac e executar com java, usando somente o JDK.

RT02. Todas as classes devem pertencer ao pacote pedidos.

RT03. Valores monetários devem usar BigDecimal. O uso de double ou float para dinheiro invalida o requisito.

RT04. Nos requisitos RF01 a RF11, a cópia deve ser implementada manualmente. Não use Cloneable nem Object.clone().

RT05. O método main não deve conter regras de negócio, apenas o cenário de teste e as verificações.

5. Cenário de teste (RF10)

Pedido-modelo (cliente "(modelo)", endereço "-"), registrado no catálogo com a chave "reposicao-mensal":

Produto	Quantidade	Preço unitário (R$)	Subtotal (R$)
Café 500g	10	18,90	189,00
Açúcar 1kg	20	4,50	90,00
Filtro de papel	5	7,20	36,00
Total do modelo			315,00

Pedidos a gerar com criar("reposicao-mensal"), nesta ordem, aplicando o ajuste indicado somente na cópia:

Pedido	Cliente	Endereço	Ajuste na cópia	Total (R$)
A	Mercado Central	Rua das Flores, 120	Nenhum ajuste.	315,00
B	Padaria Boa Massa	Av. Paraná, 800	Alterar a quantidade de Café 500g para 20.	504,00
C	Restaurante do Porto	Rua do Cais, 45	Adicionar o item Adoçante 100ml, quantidade 12, preço 6,00.	387,00

6. Critérios de aceitação (RF11)

ID	Critério
CA-01	Os quatro pedidos (modelo, A, B e C) possuem ids diferentes entre si.
CA-02	Após todas as alterações, o valor total do modelo continua R$ 315,00.
CA-03	Os totais dos pedidos são: A = R$ 315,00; B = R$ 504,00; C = R$ 387,00.
CA-04	O modelo continua com 3 itens depois de o item Adoçante ser adicionado ao pedido C.
CA-05	Nenhum ItemPedido dos pedidos A, B e C é a mesma instância (==) de um item do modelo.
CA-06	A chamada criar("inexistente") lança IllegalArgumentException.
CA-07	O cliente do modelo permanece "(modelo)" depois de os clientes das cópias serem alterados.

7. Estrutura sugerida

```
pedidos/
    Prototype.java
    ItemPedido.java
    Pedido.java
    CatalogoDePedidos.java
    PedidosApp.java
```

Para compilar e executar, a partir da pasta que contém o diretório pedidos:

```
javac -d out pedidos/*.java
java -cp out pedidos.PedidosApp
```

Referências para consulta. Gamma et al. (1994), Design Patterns, capítulo sobre Prototype; Bloch (2018), Effective Java, Item 13; https://refactoring.guru/design-patterns/prototype